



CS - 329
Operating System

Team Members:

Tuba Naushad CS-22021

Khadija Sehar CS-23014

Tahir Ali CS-23058

Submitted to:

Miss Zareen Sadiq

&

Miss Urooj Ainuddin

TABLE OF CONTENTS

Fruit Picking Synchronization Problem with GTK GUI Visualization.....	3
1. Introduction.....	3
2. Problem Description.....	3
3. Synchronization Design.....	3
3.1 Shared Resources.....	3
3.2 Synchronization Mechanisms.....	3
3.3 Crate State Machine.....	4
4. Algorithm Design.....	4
4.1 Picker Thread Logic.....	4
4.2 Loader Thread Logic.....	4
4.3 Thread Lifecycle.....	4
4.4 Inter-Thread Communication Summary.....	4
4.5 System Logic Flowchart.....	5
5. Implementation Details.....	6
6. Code.....	6
7. Test Cases.....	6
8. Execution Snapshots.....	8
Program Start.....	8
Picker Activity.....	8
Loader Activity.....	8
Final Summary Output.....	8
9. Complexity Analysis.....	9
9.1 Mutex and Condition Variable Usage.....	9
9.2 Race Condition Prevention.....	9
9.3 Deadlock & Starvation Analysis.....	9
9.4 Edge Cases Handled.....	10
9.6 Limitations.....	10
10. Conclusion.....	10
11. Future Enhancements.....	10

Fruit Picking Synchronization Problem with GTK GUI Visualization

1. Introduction

This project implements a classic producer-consumer synchronization problem (a model where some threads produce data and others consume it) using POSIX threads in C programming language. The system simulates a real-world fruit harvesting scenario involving multiple worker threads (pickers) and a loader thread responsible for transporting crates. Additionally, a GTK-based GUI is implemented to visualize thread activity, crate status, and system behavior in real time.

The goal is to ensure:

- Safe concurrent access to shared resources where multiple threads work without conflict
- Proper synchronization between threads (ensuring correct execution order)
- Fair workload distribution among pickers
- No race conditions, deadlocks (infinite waiting), or starvation (no execution chance)

2. Problem Description

The system consists of:

- **3 Picker Threads (Producers)**
Pick fruits from a shared tree and place them into a crate.
- **1 Loader Thread (Consumer)**
Waits until a crate is full, then loads it onto a truck and replaces it.

2.1 Constraints

- Each crate has a fixed capacity of 12 fruits
- Pickers can only place fruit when crate is OPEN (available)
- Loader acts only when crate becomes FULL (12/12 slots)
- Final partially filled crate must also be handled
- Fairness must be ensured among pickers

3. Synchronization Design

3.1 Shared Resources

- Tree (array of fruits)
- Crate (slots, state)
- Counters (next fruit index, crate count, etc.)

3.2 Synchronization Mechanisms

Mutexes (thread locks)

- `tree_mutex` → protects fruit picking
- `crate_mutex` → protects crate operations

Condition Variables (wait/notify mechanism)

- `cond_crate_open` → pickers wait if crate unavailable (pause)
- `cond_crate_full` → loader waits for full crate (until notified)

3.3 Crate State Machine

OPEN → FULL → SWAPPING → OPEN

State	Description
OPEN	Pickers can add fruit
FULL	Crate filled (12/12)
SWAPPING	Loader replacing crate

4. Algorithm Design

4.1 Picker Thread Logic

- Lock `tree_mutex` to pick fruit.
- Unlock and move to the crate.
- Lock `crate_mutex` and wait if not OPEN.
- Place fruit; if 12th fruit, signal loader.

4.2 Loader Thread Logic

- Lock `crate_mutex` and wait for FULL state or all pickers to finish.
- If full, mark the state as SWAPPING and load onto the truck.
- Replace with a new crate and broadcast `cond_crate_open` to all pickers.
- Handle any final partial crates before exiting.

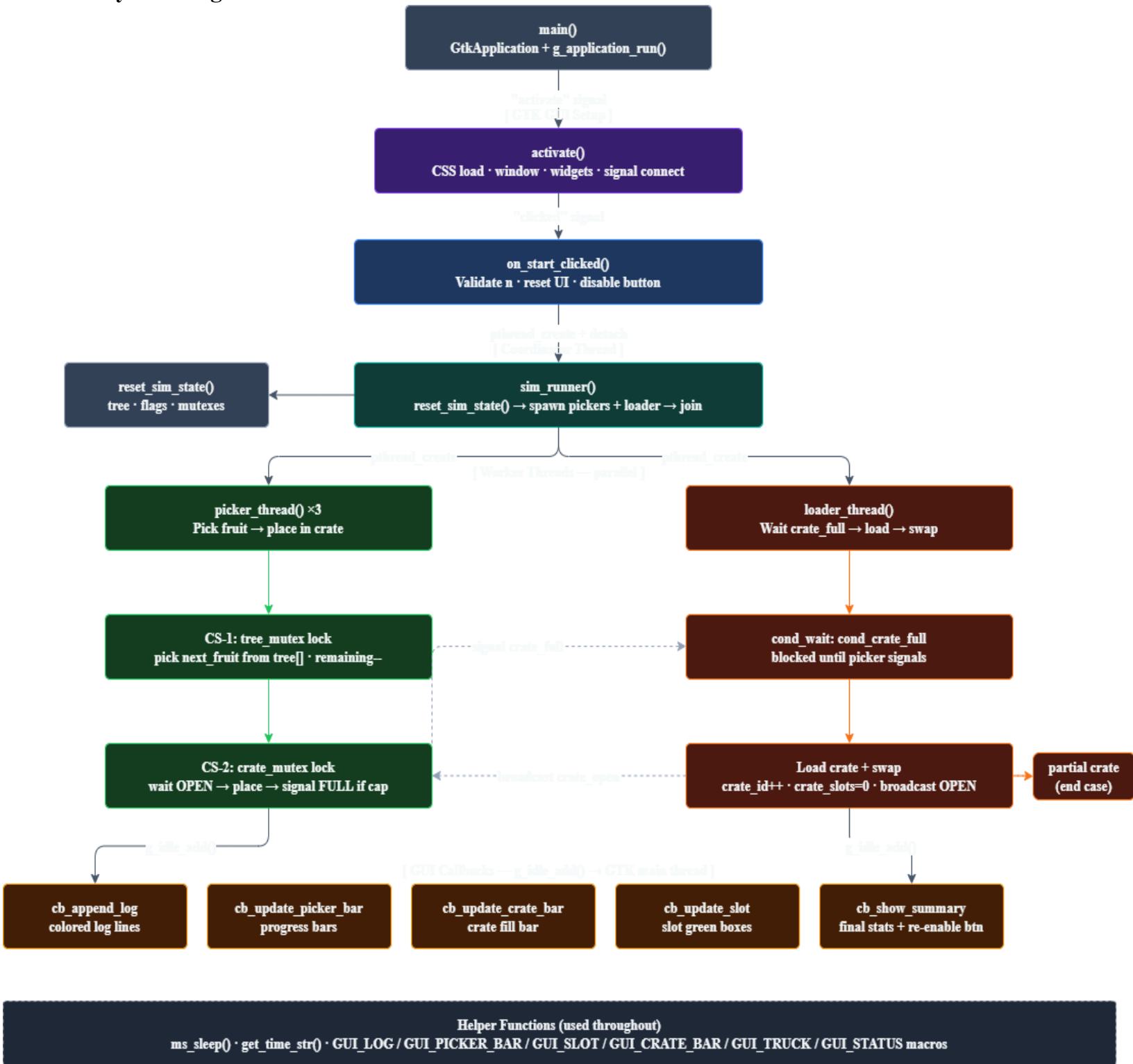
4.3 Thread Lifecycle

- Born → Spawned via `pthread_create()`.
- Running → Picking fruit or loading the crate.
- Blocked → Waiting on a mutex or `pthread_cond_wait()`.
- Signaled → Awakened via `pthread_cond_signal()` or `broadcast()`.
- Terminated → Exiting via `pthread_join()` after the tree is bare.

4.4 Inter-Thread Communication Summary

Source (From)	Destination (To)	Signal Type	Reason / Trigger
Picker	Loader	<code>cond_signal</code>	Crate full
Loader	All Pickers	<code>cond_broadcast</code>	New crate ready
Last Picker	Loader	<code>cond_signal</code>	Tree bare

4.5 System Logic Flowchart



5. Implementation Details

- **Language:** C
- **Standard:** POSIX
- **Compiler:** gcc
- **Thread Library:** pthread
- **GUI Library:** GTK+ 3 (used for visualization)
- **Signal Handling:** signal.h for SIGINT graceful exit
- **Timing Library:** clock_gettime(CLOCK_MONOTONIC) for performance metrics
- **GUI Thread Safety:** g_idle_add() ensures all widget updates happen on the main thread via heap-allocated callback structs, preventing GTK threading violations.
- **Compilation:** gcc -o Threads_GTK Threads_GTK.c -lpthread \$(pkg-config --cflags --libs gtk+-3.0)
- **Execution:** ./Threads_GTK

6. Code

Link: <https://drive.google.com/file/d/1hfDjVH4u2tzQiQga9v6ojDeebt070aA1/view?usp=sharing>

7. Test Cases

Test Case 1

Input: Number of fruits = 15

Expected Behavior:

→ Crates Loaded: 1 full (12) + 1 partial (3 fruits)

→ Loader handles partial crate at end

→ Equal distribution among pickers

Output Snapshot:

The screenshot displays the program's execution output, divided into two main sections: an 'Activity Log' on the left and a 'Picker Progress' GUI window on the right.

Activity Log: This section shows a detailed log of events, including fruit picking, crate filling, and crate loading. Key events include:

- Pickers 1, 2, and 3 picking fruits and filling slots in Crate-1.
- Crate-1 becoming full (12/12 slots filled) and being loaded onto the truck.
- A new empty Crate-2 being provided.
- Pickers 1, 2, and 3 picking the remaining 3 fruits and filling slots in Crate-2.
- Crate-2 becoming partially full (3/12 slots filled) and being loaded onto the truck.
- The loader signaling that all pickers are done and the tree is now bare.
- The loader loading the partial Crate-2 and signaling that all is done.

Picker Progress GUI: This window provides a visual summary of the progress:

- Picker Progress:** Shows three progress bars for Picker-1, Picker-2, and Picker-3, each indicating 5 fruits picked.
- Current Crate:** Shows a progress bar for Crate-2, indicating 3/12 slots filled.
- Truck:** Shows '2 crate(s) loaded'.
- Summary:** A table summarizing the overall performance:

SUMMARY	
Fruits on tree	: 15
Fruits picked	: 15
Crates on truck	: 2
Picker-1 picked	: 5
Picker-2 picked	: 5
Picker-3 picked	: 5
Correctness	: PASSED
Fairness spread	: 0 fruit(s)
Avg crate time	: 32558.61 ms

Test Case 2

Input: Number of fruits = 25

Expected Behavior:

→ Crates Loaded: 2 full (24) + 1 partial (1 fruit)

→ Loader handles partial crate at end

Output Snapshot:

Activity Log

```
[15:32:07.385] Picker-3 Picked fruit #15 (val=65) | remaining: 10
[15:32:07.635] Picker-3 Placed into Crate-2 | Slot 3/12 filled.
[15:32:07.636] Picker-1 Picked fruit #16 (val=59) | remaining: 9
[15:32:07.636] Picker-2 Picked fruit #17 (val=33) | remaining: 8
[15:32:07.886] Picker-1 Placed into Crate-2 | Slot 4/12 filled.
[15:32:07.886] Picker-2 Placed into Crate-2 | Slot 5/12 filled.
[15:32:08.116] Picker-3 Picked fruit #18 (val=75) | remaining: 7
[15:32:08.366] Picker-3 Placed into Crate-2 | Slot 6/12 filled.
[15:32:08.367] Picker-1 Picked fruit #20 (val=72) | remaining: 5
[15:32:08.367] Picker-2 Picked fruit #19 (val=74) | remaining: 6
[15:32:08.618] Picker-1 Placed into Crate-2 | Slot 7/12 filled.
[15:32:08.618] Picker-2 Placed into Crate-2 | Slot 8/12 filled.
[15:32:08.848] Picker-3 Picked fruit #21 (val=88) | remaining: 4
[15:32:09.098] Picker-3 Placed into Crate-2 | Slot 9/12 filled.
[15:32:09.099] Picker-1 Picked fruit #22 (val=73) | remaining: 3
[15:32:09.099] Picker-2 Picked fruit #23 (val=6) | remaining: 2
[15:32:09.349] Picker-1 Placed into Crate-2 | Slot 10/12 filled.
[15:32:09.349] Picker-2 Placed into Crate-2 | Slot 11/12 filled.
[15:32:09.430] Picker-2 Done. (Picked 8 fruits)
[15:32:09.579] Picker-3 Picked fruit #24 (val=21) | remaining: 1
[15:32:09.830] Picker-3 Placed into Crate-2 | Slot 12/12 filled.
[15:32:09.830] Picker-3 Crate-2 FULL! Signalling loader...
[15:32:09.830] Picker-1 Picked fruit #25 (val=4) | remaining: 0
[15:32:09.830] Loader Loading FULL Crate-2 onto truck...
[15:32:10.431] Loader Crate-2 loaded. Truck total: 2 crate(s).
[15:32:10.783] Loader New empty Crate-3 provided.
[15:32:10.783] Picker-3 New Crate-3 received. Resuming.
[15:32:10.863] Picker-3 Done. (Picked 8 fruits)
[15:32:11.034] Picker-1 Placed into Crate-3 | Slot 1/12 filled.
[15:32:11.114] Picker-1 Done. (Picked 9 fruits)
[15:32:11.114] Picker-1 Tree is now BARE. Signalling loader.
[15:32:11.114] Loader All pickers done. Loading PARTIAL Crate-3 (1/12)...
[15:32:11.715] Loader Partial Crate-3 on truck. Total: 3 crate(s).
[15:32:11.715] Loader All done. Exiting.
```

Picker Progress
Picker-1: 9 fruits
Picker-2: 8 fruits
Picker-3: 8 fruits

Current Crate
Crate-3: 1/12


Truck: 3 crate(s) loaded

Summary

SUMMARY	
Fruits on tree	: 25
Fruits picked	: 25
Crates on truck	: 3
Picker-1 picked	: 9
Picker-2 picked	: 8
Picker-3 picked	: 8
Correctness	: PASSED
Fairness spread	: 1 fruit(s)
Avg crate time	: 178416.88 ms

Test Case 3

Input: Number of fruits = 50

Expected Behavior:

→ Multiple crates loaded 4 full (48) + 1 partial (2 fruits)

→ Loader handles partial crate at end

Output Snapshot:

Activity Log

```
[15:33:42.711] Picker-2 Placed into Crate-4 | Slot 3/12 filled.
[15:33:42.711] Picker-1 Picked fruit #41 (val=9) | remaining: 9
[15:33:42.962] Picker-3 Placed into Crate-4 | Slot 4/12 filled.
[15:33:42.962] Picker-1 Placed into Crate-4 | Slot 5/12 filled.
[15:33:43.192] Picker-2 Picked fruit #42 (val=84) | remaining: 8
[15:33:43.442] Picker-2 Placed into Crate-4 | Slot 6/12 filled.
[15:33:43.442] Picker-1 Picked fruit #44 (val=11) | remaining: 6
[15:33:43.442] Picker-3 Picked fruit #43 (val=23) | remaining: 7
[15:33:43.693] Picker-3 Placed into Crate-4 | Slot 7/12 filled.
[15:33:43.693] Picker-1 Placed into Crate-4 | Slot 8/12 filled.
[15:33:43.923] Picker-2 Picked fruit #45 (val=53) | remaining: 5
[15:33:44.174] Picker-1 Picked fruit #46 (val=94) | remaining: 4
[15:33:44.174] Picker-3 Picked fruit #47 (val=4) | remaining: 3
[15:33:44.174] Picker-2 Placed into Crate-4 | Slot 9/12 filled.
[15:33:44.424] Picker-3 Placed into Crate-4 | Slot 10/12 filled.
[15:33:44.424] Picker-1 Placed into Crate-4 | Slot 11/12 filled.
[15:33:44.654] Picker-2 Picked fruit #48 (val=34) | remaining: 2
[15:33:44.905] Picker-3 Picked fruit #49 (val=54) | remaining: 1
[15:33:44.905] Picker-2 Placed into Crate-4 | Slot 12/12 filled.
[15:33:44.905] Picker-2 Crate-4 FULL! Signalling loader...
[15:33:44.905] Picker-1 Picked fruit #50 (val=18) | remaining: 0
[15:33:44.905] Loader Loading FULL Crate-4 onto truck...
[15:33:45.505] Loader Crate-4 loaded. Truck total: 4 crate(s).
[15:33:45.856] Loader New empty Crate-5 provided.
[15:33:45.856] Picker-2 New Crate-5 received. Resuming.
[15:33:45.936] Picker-2 Done. (Picked 16 fruits)
[15:33:46.106] Picker-3 Placed into Crate-5 | Slot 1/12 filled.
[15:33:46.107] Picker-1 Placed into Crate-5 | Slot 2/12 filled.
[15:33:46.187] Picker-3 Done. (Picked 17 fruits)
[15:33:46.187] Picker-1 Done. (Picked 17 fruits)
[15:33:46.187] Picker-1 Tree is now BARE. Signalling loader.
[15:33:46.187] Loader All pickers done. Loading PARTIAL Crate-5 (2/12)...
[15:33:46.787] Loader Partial Crate-5 on truck. Total: 5 crate(s).
[15:33:46.787] Loader All done. Exiting.
```

Picker Progress
Picker-1: 17 fruits
Picker-2: 16 fruits
Picker-3: 17 fruits

Current Crate
Crate-5: 2/12

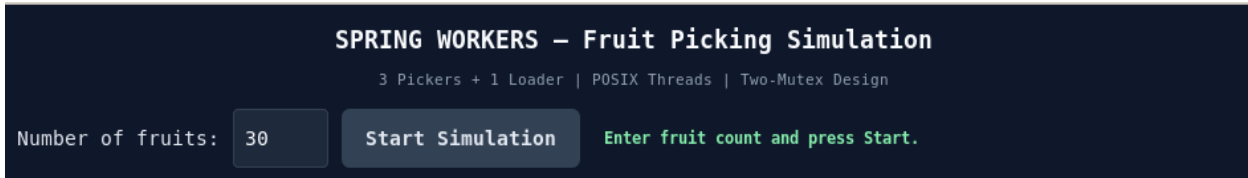

Truck: 5 crate(s) loaded

Summary

SUMMARY	
Fruits on tree	: 50
Fruits picked	: 50
Crates on truck	: 5
Picker-1 picked	: 17
Picker-2 picked	: 16
Picker-3 picked	: 17
Correctness	: PASSED
Fairness spread	: 1 fruit(s)
Avg crate time	: 21514.02 ms

8. Execution Snapshots

Program Start



Picker Activity

```
[16:20:37.742] Picker-1 Picked fruit #1 (val=59) | remaining: 29
[16:20:37.746] Picker-3 Picked fruit #2 (val=61) | remaining: 28
[16:20:37.749] Picker-2 Picked fruit #3 (val=39) | remaining: 27
[16:20:37.993] Picker-1 Placed into Crate-1 | Slot 1/12 filled.
[16:20:38.003] Picker-2 Placed into Crate-1 | Slot 2/12 filled.
[16:20:38.037] Picker-3 Placed into Crate-1 | Slot 3/12 filled.
```

Loader Activity

```
[16:20:40.251] Picker-3 Crate-1 FULL! Signalling loader...
[16:20:40.251] Loader Loading FULL Crate-1 onto truck...
[16:20:40.686] Picker-1 Picked fruit #13 (val=80) | remaining: 17
[16:20:40.696] Picker-2 Picked fruit #14 (val=38) | remaining: 16
[16:20:40.851] Loader Crate-1 loaded. Truck total: 1 crate(s).
[16:20:41.202] Loader New empty Crate-2 provided.
```

Final Summary Output

```
Truck: 9 crate(s) loaded

Summary
===== SUMMARY =====
Fruits on tree : 100
Fruits picked : 100
Crates on truck : 9
Picker-1 picked : 33
Picker-2 picked : 34
Picker-3 picked : 33
Correctness : PASSED
Fairness spread : 1 fruit(s)
Avg crate time : 14855.88 ms
```

```
Truck: 1 crate(s) loaded

Summary
===== SUMMARY =====
Fruits on tree : 2
Fruits picked : 2
Crates on truck : 1
Picker-1 picked : 1
Picker-2 picked : 1
Picker-3 picked : 0
Correctness : PASSED
Fairness spread : 1 fruit(s)
Avg crate time : 0.00 ms
```

9. Complexity Analysis

9.1 Mutex and Condition Variable Usage

- **tree_mutex:** Protects the fruit count. It is held only for a split second to increment the counter.
- **crate_mutex:** Protects the crate slots. It ensures only one picker writes to a slot at a time.
- **Design Rule:** Both mutexes are *never* held simultaneously. This simple rule completely prevents deadlocks. Two separate condition variables handle signals between pickers and the loader without confusion.
- **cond_crate_open:** Pickers wait on this variable when *crate_state* != *CRATE_OPEN*. The picker sleeps here releasing *crate_mutex* automatically until the loader calls *pthread_cond_broadcast(&cond_crate_open)*. This ensures no CPU cycles are wasted during the wait.
- **cond_crate_full:** The loader waits on this variable until either a picker signals that the crate is full via *pthread_cond_signal(&cond_crate_full)*, or the final picker signals that the tree is bare via *pickers_done* == *NUM_PICKERS*.
- **Separation of Concerns:** This two-condition-variable design cleanly separates communication channels: pickers notify the loader through *cond_crate_full*, while the loader notifies pickers through *cond_crate_open*.

9.2 Race Condition Prevention

- **Double-picking:** The mutex prevents two pickers from grabbing the same fruit index.
- **Slot Overlap:** The mutex ensures no two pickers try to put a fruit into the same crate slot.
- **Early Loading:** The loader waits for the *CRATE_FULL* status, ensuring it doesn't empty the crate while a picker is still placing a fruit.
- **Completion Handshake:** The loader's use of *pickers_done* counter ensures that no fruit is left in limbo when the tree becomes bare, only after all pickers have finished does the loader process the final partial crate.
- **Performance Tracking:** The system tracks average crate filling time using *clock_gettime(CLOCK_MONOTONIC)*. Each time the loader provides a new crate, a timer starts. When the 12th slot is filled, elapsed time is recorded in *total_crate_time* and averaged across all crates. This metric appears in the final GUI summary as "*Avg crate time*" in milliseconds, helping identify picker throughput.
- **Slot Overflow Prevention:** The *CRATE_FULL* state is set inside *crate_mutex* before unlocking, ensuring no other picker can enter the crate between the state change and the unlock. This closes a subtle race window that would otherwise allow slot overflow.

9.3 Deadlock & Starvation Analysis

- **Mutual Exclusion:** Mutexes ensure only one thread accesses shared resources at a time.
- **Fixed Lock Order:** *tree_mutex* is always acquired before *crate_mutex*, eliminating circular wait and deadlock risk.
- **Separate Critical Sections:** *tree_mutex* is fully released before *crate_mutex* is acquired so no thread holds both simultaneously, breaking the hold-and-wait condition.
- **Auto-Release on Wait:** *pthread_cond_wait* atomically releases the mutex while sleeping, keeping other threads unblocked.
- **Final Broadcast:** At simulation end, a broadcast wakes any remaining waiting threads to ensure clean exit.

9.4 Edge Cases Handled

- **Perfect Multiples:** If the fruit count is a multiple of 12, the loader exits cleanly without trying to process an empty 13th crate.
- **Empty Tree Mid-Task:** If the tree runs out of fruit while a picker is working, they are allowed to finish placing their last fruit before exiting.
- **Fault Tolerance:** A *SIGINT* handler registered via *signal(SIGINT, handle_sigint)* ensures that if the simulation is force-closed with *Ctrl+C*, both mutexes are unlocked, all waiting threads are broadcast, and tree memory is freed before exit preventing resource leaks and deadlocks on abnormal termination.

9.6 Limitations

- **Bottleneck:** Since there is only one crate, pickers must wait for each other, which slows down the process.
- **Single Failure Point:** If the loader thread crashes or hangs, the entire system stops.
- **Memory:** Very large fruit counts could crash the program because there is no limit on memory allocation (*malloc*).
- **Compatibility:** The code is designed for Linux (POSIX/GTK) and will not run on Windows without major changes.

10. Conclusion

This project successfully implements a multi-threaded fruit harvesting simulation using POSIX threads, mutexes, and condition variables in C. Three picker threads and one loader thread run in parallel, safely sharing a tree and crate without race conditions, deadlocks, or starvation. Beyond basic synchronization, the implementation includes real-time performance tracking via *clock_gettime*, graceful fault tolerance via *SIGINT* handling, and GUI thread safety via *g_idle_add* bringing it closer to a concurrent system design that reflects real-world operating system challenges.

11. Future Enhancements

- **Runtime Configuration:** Make *NUM_PICKERS* and *CRATE_CAPACITY* adjustable via GUI sliders, no recompilation needed.
- **Parallel Crates:** Give each picker its own buffer crate to eliminate the placement bottleneck, only the loader touches the shared truck-bound crate.
- **Timeout on Waits:** Replace *pthread_cond_wait* with *pthread_cond_timedwait* so pickers exit gracefully if the loader hangs instead of freezing forever.
- **Speed Control:** Add a real-time slider to adjust *DELAY_PICK_MS*, *DELAY_PLACE_MS*, and *DELAY_LOAD_MS* for better observation of thread behavior.
- **Warehouse Scaling:** Support N pickers and M parallel crates via a lock-free queue, enabling warehouse-level simulations without restructuring core synchronization logic.
- **Priority Inversion Protection:** Initialize *crate_mutex* with *PTHREAD_PRIO_INHERIT* via *pthread_mutexattr_set* protocol to prevent a high-priority loader from being delayed by a lower-priority lock-holding picker.